Pseudorandom Numbers

- There are several functions in Python that return ***pseudorandom*** numbers.
- The numbers are called ***pseudorandom*** numbers because although they are not truly random they appear to be (that is they are very close to being random).
- There should be a uniform distribution of random numbers. That is if you are generating random numbers between 0 and 1 the numbers that are generated should be evenly distributed between 0 and 1.

Pseudorandom Numbers

- Pseudorandom numbers can be used to model probabilities.
- If you roll a die what is the probability of rolling a 6?
- A die has six values, 1, 2, 3, 4, 5 and 6. If it is equally likely that each of the values can be rolled, then there is a 1 in 6 chance that a 6 will be rolled.
- For each of the six values there is a 1 in 6 chance the value will be rolled.

- The function ***random*** in the module ***random*** generates pseudorandom numbers as floats from 0 up to but **not** including 1.
- The notation for this is [0, 1) and is known as a half open interval.
- To generate random integers between 1 and 6 inclusive:

```
# rollDie.py

from random import random

def rollDie():
    return int(random() * 6) + 1

print('The number rolled is', rollDie())
```

x = random()	0. <= x < 1.
x = random() * 6	0. <= x < 6.
x = int(random() * 6)	0 <= x <= 5
x = int(random() * 6) + 1	1 <= x <= 6

- Output from the program might be:

```
The number rolled is 4
```

- Suppose we would like to determine the probability of rolling each of the numbers between 1 and 6 inclusive.
- Roll the die a large number of times and count the number of times each number comes up. Divide this by the number of rolls to get the probabilities.

```
# rollDieProbability.py

from random import random

def rollDie():
    return int(random() * 6) + 1

def main():
    rolls = [0] * 6
    numRolls = int(input('Enter the number of times'+\
                  ' to roll the die (> 0): '))
    if numRolls > 0:
        for i in range(numRolls):
            rolls[rollDie()-1] += 1
        for roll in range(6):
            probability = rolls[roll] / numRolls
            print('roll: %d, probability: %.3f' % \
                  (roll+1, probability))
    else:
        print(numRolls, ' is not greater than zero!')

main()
```

•Output from the program:

```
Enter the number of times to roll the die (>
0): 600000
roll: 1, probability: 0.167
roll: 2, probability: 0.167
roll: 3, probability: 0.167
roll: 4, probability: 0.167
roll: 5, probability: 0.167
roll: 6, probability: 0.166
```

•The function ***uniform(a,b)*** in the module ***random*** returns random floats starting at ***a*** but less than ***b***.

```
In [1]: import random
In [2]: print(random.uniform(0,5))
2.0078211947375326
```

•Rewrite the program to use the *uniform* function.

```
# rollDieProbability1.py

from random import uniform

def rollDie():
    return int(uniform(1, 7))

def main():
    rolls = [0] * 6
    numRolls = int(input('Enter the number of times to' +\
                         ' roll the die (> 0): '))
    if numRolls > 0:
        for i in range(numRolls):
            rolls[rollDie()-1] += 1
        for roll in range(6):
            probability = rolls[roll] / numRolls
            print('roll: %d, probability: %.3f' \
                  % (roll+1, probability))
    else:
        print(numRolls, ' is not greater than zero!')

main()
```

•Output from the program:

```
Enter the number of times to roll the die (>
0): 600000
roll: 1, probability: 0.166
roll: 2, probability: 0.165
roll: 3, probability: 0.167
roll: 4, probability: 0.166
roll: 5, probability: 0.167
roll: 6, probability: 0.168
```

•The ***randint(a, b)*** function in the ***random*** module returns integers in the range ***a*** to ***b*** inclusive.

•Rewrite the program to use the ***randint*** function.

```
# rollDieProbability2.py

from random import randint

def rollDie():
    return randint(1, 6)

def main():
    rolls = [0] * 6
    numRolls = int(input('Enter the number of times to '+\
                         'roll the die (> 0): '))
    if numRolls > 0:
        for i in range(numRolls):
            rolls[rollDie()-1] += 1
        for roll in range(6):
            probability = rolls[roll] / numRolls
            print('roll: %d, probability: %.3f'\
                  % (roll+1, probability))
    else:
        print(numRolls, ' is not greater than zero!')

main()
```

•The output from the program:

```
Enter the number of times to roll the die (>
0): 600000
roll: 1, probability: 0.167
roll: 2, probability: 0.166
roll: 3, probability: 0.167
roll: 4, probability: 0.167
roll: 5, probability: 0.167
roll: 6, probability: 0.166
```

•Suppose you roll a pair of dice, what is the probability of rolling each of the 11 possible values, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 and 12?

	2	3	4	5	6	7
	3	4	5	6	7	8
	4	5	6	7	8	9
	5	6	7	8	9	10
	6	7	8	9	10	11
	7	8	9	10	11	12

The number of times a value occurs in the table given above divided by the number of the entries in the table gives the probability of the value. There are 36 enters in the table.
7 occurs 6 times in the table so the probability of rolling a 7 is 6/36 or 0.167

```
# rollDiceProbability.py

from random import randint

rollDie = lambda : randint(1, 6)

rollDice = lambda : rollDie() + rollDie()

def main():
    rolls = [0] * 11
    numRolls = int(input('Enter the number of times '+\
                         'to roll the die (> 0): '))
    if numRolls > 0:
        for i in range(numRolls):
            rolls[rollDice()-2] += 1
        for roll in range(11):
            probability = rolls[roll] / numRolls
            print('roll: %2d, probability: %.3f' \
                  % (roll+2, probability))
    else:
        print(numRolls, ' is not greater than zero!')

main()
```

•The output from this program is:

```
Enter the number of times to roll the die (>
0): 360000
roll:  2, probability: 0.028
roll:  3, probability: 0.055
roll:  4, probability: 0.083
roll:  5, probability: 0.111
roll:  6, probability: 0.139
roll:  7, probability: 0.166
roll:  8, probability: 0.140
roll:  9, probability: 0.111
roll: 10, probability: 0.082
roll: 11, probability: 0.056
roll: 12, probability: 0.028
```

- Often we would like to plot information related to random numbers.
- numpy contains functions that generate pseudorandom numbers in the ***numpy.random*** module.
- This modules has functions similar to those in ***random*** except they generate arrays and accept the number of random numbers to put in the arrays.

```
In [1]: import numpy as np
In [2]: np.random.randint(1, 7, 20)
array([3, 5, 4, 2, 4, 2, 3, 4, 4, 5, 2, 5, 1,
1, 3, 5, 2, 5, 1, 6])
```

- Notice that the upper bound 7 is not included in the random integers that are generated.
- 20 specifies the number of random integers to generate.

```
In [1]: np.random.uniform(1, 6, 20)
array([ 5.37910625,  4.80609278,  3.45094463,
3.48421878,  2.54784825,  1.59434271,
1.49500393,  2.6263714 ,  4.09955783,
5.19852255,  3.56174217,  1.70983072,
1.13820888,  4.90404528,  3.80039729,
2.82095192,  1.45942938,  5.26220993,
4.6017007 ,  4.12015012])

In [2]: np.random.random(20)
array([ 0.46154563,  0.67647958,  0.49041033,
0.00276733,  0.76114292,  0.5927015 ,
0.70847449,  0.2801141 ,  0.86451295,
0.36072115,  0.57178297,  0.47580318,
0.86339528,  0.40329832,  0.00715072,
0.44211294,  0.35458592,  0.19312487,
0.96676584,  0.73843041])
```

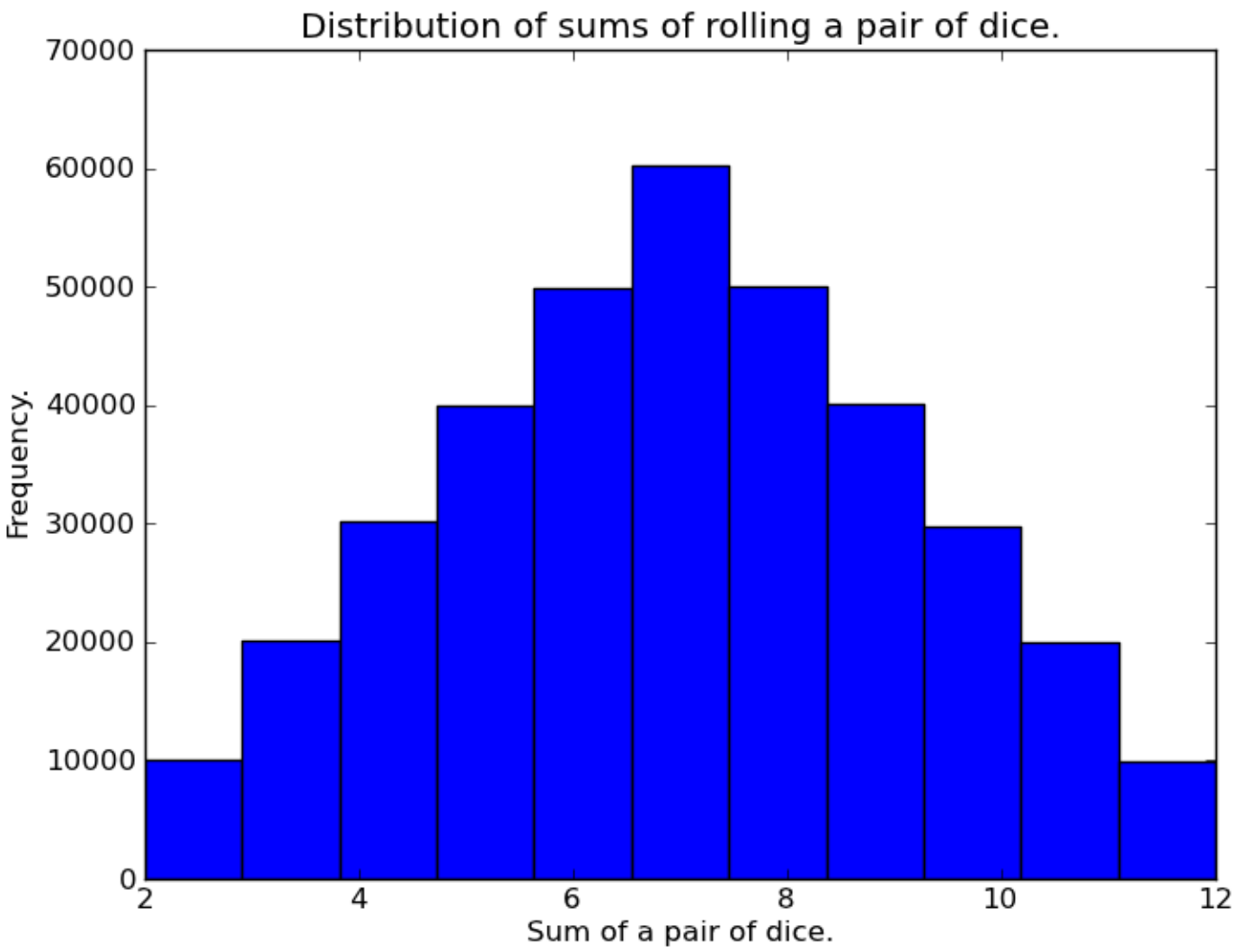
•Display a histogram for the numbers generated by rolling a pair of dice.

```
# rollDiceProbability2.py
import numpy as np
import matplotlib.pyplot as plt

rollDie = lambda numRolls: np.random.randint(1, 7, numRolls)
rollDice = lambda numRolls: rollDie(numRolls) + rollDie(numRolls)

def main():
    numRolls = int(input('Enter the number of times to roll '+\
                         'the die (> 0): '))
    if numRolls > 0:
        rolls = rollDice(numRolls)
        plt.hist(rolls, bins=11)
        plt.title('Distribution of sums of rolling a pair ' +\
                  'of dice.')
        plt.xlabel('Sum of a pair of dice.')
        plt.ylabel('Frequency.')
        plt.savefig('DiceDistro.png')
        plt.show()
    else:
        print(numRolls, ' is not greater than zero!')

main()
```


Distribution of sums of rolling a pair of dice.
70000
60000
50000
40000
30000
20000
10000
0
Frequency.
2
4
6
8
10
12
Sum of a pair of dice.

- Write a program that determines how many people have to be in a room in order for them to have a birthday on the same day in the same month. The year of birth is ignored.
- Assume there are 365 days in a year ignoring leap years.
- What is your guess?

```
# birthdays.py

from random import randint
import numpy as np
from math import ceil

birthday = lambda days : randint(1, days)
```

```
def sameBirthday(days):
    birthdays = np.zeros((days), bool)
    count = 0
    while True:
        day = birthday(days)-1
           count += 1
        if birthdays[day]:
            break
        else:
            birthdays[day] = True
 return count
```

```
def main():
    days = 365
    count = 0
    samples = 10000
    for i in range(samples):
        count += sameBirthday(days)
    people = ceil(count/samples)
    print('The number of people required to '+
          'have the same birthday is %d.'
          % people)

main()
```

- Assume that there are 365 people in a room and that a year has 365 days.
- As before the year of birth is ignored.
- How many different birthdays will the 365 people have?

```
# differentBirthdays.py

from random import randint
import numpy as np

birthday = lambda days : randint(1, days)
```

```
def differentBirthdays(people, days):
    birthdays = np.zeros((days), bool)
    for i in range(people):
        day = birthday(days)-1
        if not birthdays[day]:
            birthdays[day] = True
    # sum(birthdays) counts the number
    # of occurrences of True in the
    # array.
    return sum(birthdays)
```

```
def main():
    days = 365
    people = 365
    count = 0
    samples = 10000
    for i in range(samples):
        count += differentBirthdays(people,
                                    days)
    different = count//samples
    print('%d people have %d different' +
          ' birthdays.'
          % (people, different))


main()
```

- How many people do you need to have in a room so that at least one person has a birthday on every day of the year?
- Assume there are 365 days in a year.

```
# allBirthdays.py

from random import randint
import numpy as np
from math import ceil

birthday = lambda days : randint(1, days)
```

```
def allBirthdays(days):
    people = 0
    count = 0  # count days with birthdays
    birthdays = np.zeros((days), bool)
    while count < days:
        day = birthday(days)-1
        people += 1
        if not birthdays[day]:
            birthdays[day] = True
            count += 1
    return people
```

```
def main():
    days = 365
    count = 0
    samples = 10000
    for i in range(samples):
        count += allBirthdays(days)
    people = ceil(count/samples)
    print('It takes %d people to have ' +
          '% d different birthdays.'
          % (people, days))

main()
```

- Write a program that generates passwords that contain letters.
- Each password starts with an uppercase letter and contains only uppercase and lowercase letters.
- The passwords must contain at least 8 characters.
- We want to generate random characters between 'A' and 'Z' and between 'a' and 'z'.
- We can use ***randint*** if we convert the bounds from letters to integers.
- To do this we use the ***ord*** function.

```
In [1]: ord('A'), ord('Z'), ord('a'), ord('z')
(65, 90, 97, 122)
In [2]: print randint(ord('A'), ord('Z'))
76
```

- What letter does 76 represent?

- Use the ***chr*** function to convert the number 76 into the letter it represents as a string value.

```
In [1]: print(chr(76))
'L'
In [2]: print(type(chr(76)))
<class 'str'>
```

- To generate random uppercase letters do the following:

```
In [3]: print(chr(randint(ord('A'),
        ord('Z'))))
'W'
In [4]: print(chr(randint(ord('A'),
        ord('Z'))))
'B'
```

- The program that generates random passwords follows.

```
# passwords.py

from random import randint

def randomChar(lowerBound, upperBound):
    return chr(randint(ord(lowerBound),
                       ord(upperBound)))

randomUppercase = lambda : randomChar('A', 'Z')
randomLowercase = lambda : randomChar('a', 'z')

def randomLetter():
    return (randomLowercase() if randint(0, 1)
            == 0 else randomUppercase())
```

```
def randomPassword(length):
    password = randomUppercase()
    for i in range(length-1):
        password+=randomLetter()
    return password

def main():
    length = int(input('Enter the length of the ' +
                       'passwords (> 7): '))
    if length > 7:
        numPasswords = int(input('Enter the number '+
                                 'of passwords: '))
        print('Random passwords of length %d:'
              % length)
        for passwords in range(numPasswords):
            print(randomPassword(length))
    else:
      print('The length of the passwords must be > 7!')

main()
```

•Output from the program:

```
Enter the length of the passwords (> 7): 12
Enter the number of passwords: 10
Random passwords of length 12:
DIxAptxJisIY
YbgcHEMrATtL
FUifBIqyFmAV
IGWqgUjQNXji
AaCuxPeBMaSI
BSIgymXFforD
JNinzbsbDHBi
WWDhDSDEPPjU
YocjLkXWuMFG
DEOBxdrGWjbN
```

- Each time you run a program that uses random numbers you get a different answer.
- When you are **testing** a program you often want to get the same answer each time the program is run.
- To generate the same random numbers each time the program is run specify the ***seed***.
- The ***seed*** is the value from which all the random numbers will be generated.
- Normally the ***seed*** is determined from the current time, how long the computer is running, movement of the mouse, the number of keystrokes, etc.
- If you set the ***seed*** to an integer value in your program, it will produce the same sequence of random numbers each time the program is run.

```
# passwords1.py
from random import randint, seed  # added seed

def main():
    length = int(input('Enter the length of ' +
                       'the passwords (> 7): '))
    if length > 7:
        seed(42)  # added call to seed
        numPasswords = int(input('Enter the ' +
                       'number of passwords: '))
        print('Random passwords of length %d:'
              % length)
        for passwords in range(numPasswords):
            print(randomPassword(length))
    else:
        print('The length of the passwords must' +
              ' be > 7!')
```

Monte Carlo Technique

- The Monte Carlo technique uses random numbers to approximate the value of something such as, the area under a curve, the volume of a cone, etc.
- Consider finding the area of a circle whose radius is *r*.
- The formula for the area of a circle is $\boldsymbol{area = \pi r^2}$
- The smallest square that fully encloses the circle of radius *r* centered on the origin of an XY plane has
  X coordinates that range from *-r* to *+r* and
  Y coordinates that range from *-r* to *+r*.
- The area of the square is $\boldsymbol{2r \times 2r = 4r^2}$

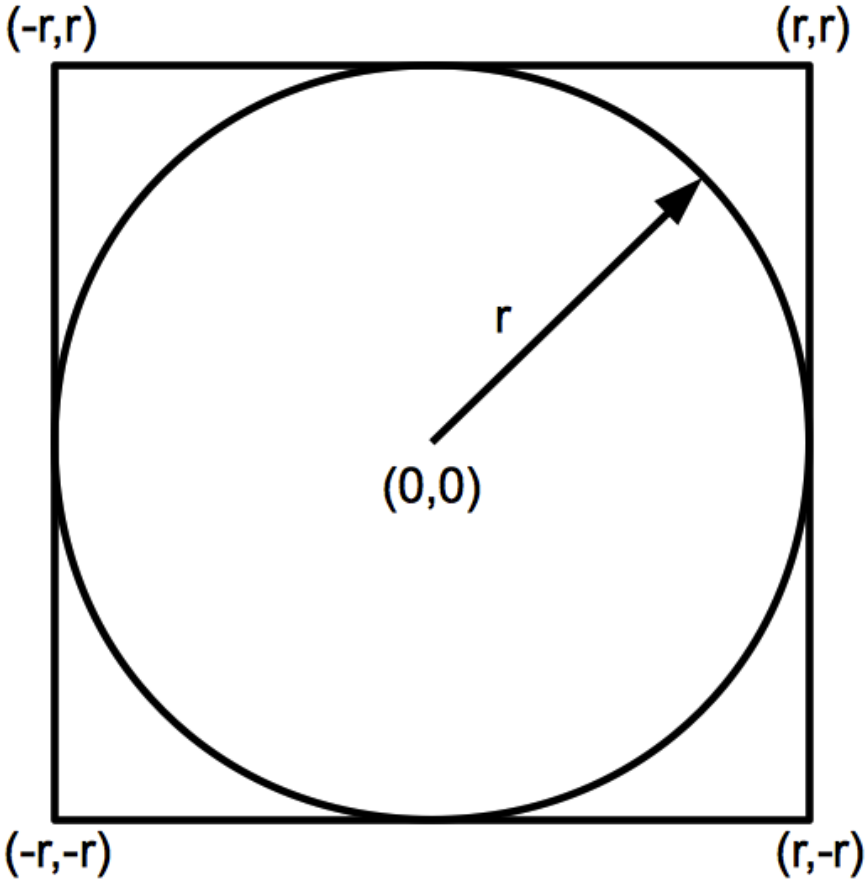


Figure 1

•To use the Monte Carlo technique to calculate the area of a circle whose radius is *r:*

–create ***n*** random points inside the square from *X = -r* to *X = r* and from *Y = -r* to *Y = r* (see Figure 1)

–count the number of points in the circle and divide by ***n*** to get the probability of a point being inside the circle

–multiply the probability of a point being inside the circle by the area of the smallest square that fully encloses the circle to get the area of the circle

•To create the random points

–create ***n*** random numbers between *-r* and *r* as the X coordinates of the points

–create ***n*** random numbers between *–r* and *r* as the Y coordinates of the points

•A point (x,y) is inside the circle if

```
from numpy.random import uniform  # MonteCircle.py
from numpy import sum, pi

r = float(input('Enter the radius of the circle: '))
if r > 0:
    points = int(input('Enter the number of points: '))
    if points > 0:
        xCoords = uniform(-r, r, points)
        yCoords = uniform(-r, r, points)
        inCircle = xCoords*xCoords + yCoords*yCoords <= r*r
        probability = sum(inCircle) / points
        area = probability * 4. * r * r
        actualArea = pi * r * r
        error = abs(area - actualArea)
        print('The probability of a point being in the circle is %.4f'
              % probability)
        print('The approximate area of the circle is %.15f' % area)
        print('     The actual area of the circle is %.15f'
              % actualArea)
        print('    The error in the approximation is % e' % error)
    else:
        print('The number of points must be greater than zero!')
else:
    print('The radius of the circle must be greater than zero!')
```

A sample run of the program with a radius of 2.5 is shown below:

```
Enter the radius of the circle: 2.5
Enter the number of points: 1000000
The probability of a point being in the circle is 0.7855
The approximate area of the circle is 19.637625000000000
     The actual area of the circle is 19.634954084936208
    The error in the approximation is  2.670915e-03
```

•Use the Monte Carlo Technique to find the volume of a pyramid.

```
from numpy.random import uniform
import numpy as np

def montePyramid(length, width, height, points):
    # xCoords - correspond to the length of the pyramid
    # yCoords - correspond to the width of the pyramid
    # zCoords - correspond to the height of the pyramid
    xCoords = uniform(0., length, points)
    yCoords = uniform(0., width, points)
    zCoords = uniform(0., height, points)
    factor = (height - zCoords) / height
    # np.all works like and for arrays
    xInPyramid = np.all([0 <= xCoords, xCoords <= length *
                 factor], axis = 0)
    yInPyramid = np.all([0 <= yCoords, yCoords <= width *
             factor], axis = 0)
    inPyramid = np.all([xInPyramid, yInPyramid], axis=0)
    probability = np.sum(inPyramid) / points
    volume = probability * length * width * height
    return probability, volume
```

•The **and** operator does not work for arrays, instead use the **all** function.

•We would like to do the following:

```
xInPyramid = 0 <= xCoords <= length * factor
yInPyramid = 0 <= yCoords <= width * factor
inPyramid = xInPyramid and yInPyramid]
```

but instead we must do the following:

```
xInPyramid = np.all([0 <= xCoords, xCoords <= length
             * factor], axis = 0)
yInPyramid = np.all([0 <= yCoords, yCoords <= width
             * factor], axis = 0)
inPyramid = np.all([xInPyramid, yInPyramid], axis=0)
```

•Similarly the **or** operator does not work for arrays, instead use the **any** function.